

Esse projeto consiste em um controle de cafeteira que permite ao usuário programar os horários em que ela liga (nessa versão, apenas 3 horários por dia, mas isso pode ser facilmente aumentado), o tempo em que ela fica ligada em cada horário. E quando ela desliga, toca uma música. A musica que eu escolhi foi uma da Lady Gaga.

A música deve ser salva na mesma pasta que o código em python e deve ter um nome padrão (que está escrito no código).

Meus vídeos no Youtube

1

Ligações

1

Códigos

3

Código ESP32

3

Código Python

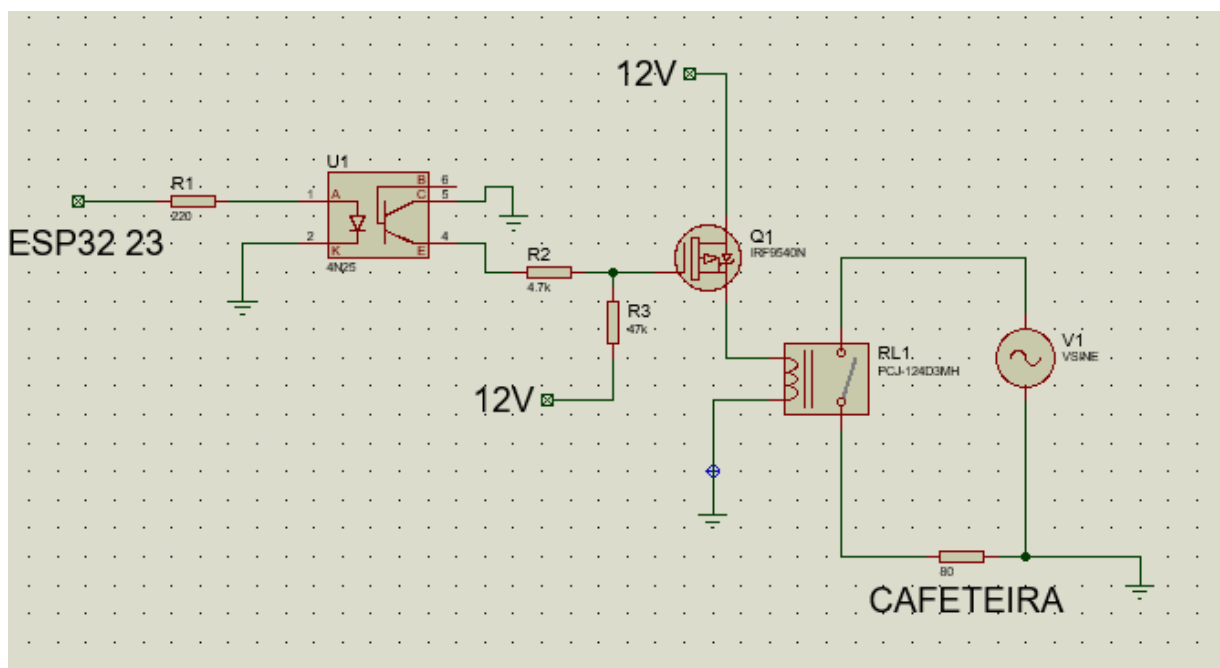
15

Meus vídeos no Youtube

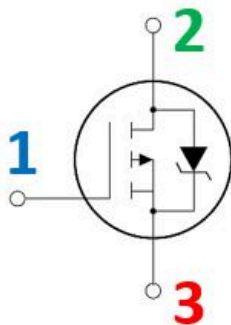
https://youtube.com/shorts/7Xxz6_hk3-Y

<https://youtube.com/shorts/cAbLUI6m0pc?feature=share>

Ligações



IRF9540 - Pinagem - PinOut



1 - Gate

2 - Dreno

3 - Source

FVM Learning

www.fvml.com.br

ESP32—OPTOACOPLADOR—PULL UP—MOSFET—RELÉ—CAFETEIRA.

Por que o ESP32 não é ligado direto à cafeteira?

Porque a cafeteira é acionada com 220V de tensão alternada e o ESP32 só consegue injetar 3,3V. Então é necessário um relé.

Por que o ESP32 não é ligado direto no relé?

Porque eu adquiri um relé de 12V e o ESP32 só consegue injetar 3,3V. Então é necessário um MOSFET.

Por que o ESP32 não é ligado direto ao MOSFET?

Primeiro para proteger o ESP32 contra erros de ligação que poderiam queimar ele (pois se injetar 12V sem querer no ESP32 ele queima) e segundo que eu preciso usar um Pull Up no MOSFET, porque ele é acionado com um potencial próximo ao terra em seu gate, mas para desacionar ele, é necessário um potencial próximo ao de 12V no seu gate. Por isso pus um divisor de tensão que quando é para acionar o MOSFET, o potencial elétrico no gate é próximo a 0V em relação ao terra, quando é para desacionar ele, o potencial é 12V no gate em relação ao terra.

Por que o ESP32 não é ligado direto ao Pull Up?

Sinceramente, não importa se funcionaria ou não. A questão é que precisa de um optoacoplador para proteger o ESP32 de erros de ligação, o optoacoplador isola um lado do outro, de forma que se ocorrer um problema no restante do circuito, o ESP32 não irá queimar.

As ligações foram feitas como acima: o ESP32 envia sinal para um optoacoplador, que aciona um transistor MOSFET de canal P (usando um divisor de tensão Pull Up), ou seja, que é próprio para ser acionado com o GND, e injeta 12V no relé, acionando a cafeteira com 220V.

Códigos

O ESP32 possui um código que se conecta com a rede wi-fi da casa, desde que seja de 2,4GHz. Para isso, em seu código, deve-se colocar a senha e nome do wi-fi. O computador deve estar conectado a mesma rede wi-fi que o ESP32.

Dessa forma, o ESP32 cria uma página web, em linguagem HTML (que é a usada em muitos sites) que permite ao usuário preencher os horários de ligar a cafeteira e etc.

Para acessar essa página WEB, é só digitar o endereço de IP do ESP32, que é informado por ele mesmo através da comunicação serial, na porta COM, logo depois de carregarmos o código nele caso não dê nenhum erro. Lamento informar, mas ela só funciona em dispositivos que estejam conectados à mesma rede wi-fi que o ESP32. Recomendo bloquear antivírus que possam impedir a comunicação. Além disso, em caso de erros, deve-se liberar a porta 8080 do computador pois pode ser que o antivírus tenha bloqueado ela (isso nas configurações avançadas do PC).

O código em Python se comunica com o ESP32, para isso, precisamos achar o endereço de IP do PC dentro da rede wi-fi. Para saber esse endereço, é só abrir o prompt de comando e digitar ipconfig, algo assim (tem diversos tutoriais na internet). Após descobrir o endereço de IP, tem que colocar isso no código do ESP32.

Código ESP32

```
#include <WiFi.h>
#include <WebServer.h>
#include <NTPClient.h>
#include <WiFiUdp.h>

// ===== CONFIGURAÇÕES =====
const char* ssid = "motorola edge 30_3562";
```

```

// motorola edge 30_3562
const char* password = "bundaoao";

// Configure o IP do computador onde o Python está rodando
const char* python_server = " 10.181.213.28"; // ALTERE PARA O IP DO
SEU PC
const int python_port = 8080;

#define RELE_PIN 23
#define TIME_ZONE_OFFSET -3
#define MAX_HORARIOS 3

WebServer server(80);
WiFiUDP ntpUDP;
NTPClient timeClient(ntpUDP, "pool.ntp.org", TIME_ZONE_OFFSET * 3600,
60000);

struct Agendamento {
    int hora;
    int minuto;
    int duracaoMinutos;
    bool ativo;
    bool executadoHoje;
    bool tocarMusica; // Nova flag para música
};

Agendamento horarios[MAX_HORARIOS];
unsigned long tempoLigado[MAX_HORARIOS] = {0, 0, 0};
bool releLigadoPorHorario[MAX_HORARIOS] = {false, false, false};

// ===== FUNÇÃO PARA NOTIFICAR PYTHON =====
void notificarPython(int horario_index) {
    WiFiClient client;

    if (!client.connect(python_server, python_port)) {
        Serial.println("Falha ao conectar ao servidor Python");
        return;
    }

    String hora_atual = timeClient.getFormattedTime();
    String postData = "horario=" + String(horario_index) +
        "&hora=" + hora_atual +
        "&status=pronto";

```

```

client.println("POST /cafe_pronto HTTP/1.1");
client.println("Host: " + String(pyhton_server));
client.println("Content-Type: application/x-www-form-urlencoded");
client.println("Connection: close");
client.print("Content-Length: ");
client.println(postData.length());
client.println();
client.println(postData);

Serial.println("Notificação enviada para Python: Café do horário " +
String(horario_index+1) + " pronto!");

delay(10);
client.stop();
}

// ===== HTML (CORRIGIDO) =====
const char INDEX_HTML[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width,
initial-scale=1">
  <title>ESP32 Cafeteira</title>
  <style>
    body {
      font-family: Arial;
      text-align: center;
      margin: 20px;
      background-color: #f0f0f0;
    }
    .container {
      max-width: 500px;
      margin: 0 auto;
      background: white;
      padding: 20px;
      border-radius: 10px;
      box-shadow: 0 0 10px rgba(0,0,0,0.1);
    }
    h1 {
      color: #333;
      margin-bottom: 20px;

```

```
}
input, button {
    padding: 8px;
    margin: 5px;
    border: 1px solid #ccc;
    border-radius: 4px;
    font-size: 16px;
}
input[type="number"] {
    width: 60px;
}
button {
    background: #4CAF50;
    color: white;
    border: none;
    cursor: pointer;
    padding: 10px 20px;
}
button:hover {
    background: #45a049;
}
.horario-box {
    border: 1px solid #ddd;
    border-radius: 5px;
    padding: 15px;
    margin: 10px 0;
    background: #f9f9f9;
}
.status {
    padding: 10px;
    margin: 10px 0;
    border-radius: 5px;
}
.pronto {
    background: #d4edda;
    color: #155724;
    border: 1px solid #c3e6cb;
}
.desligado {
    background: #f8d7da;
    color: #721c24;
    border: 1px solid #f5c6cb;
}
```

```

        .current-time {
            font-size: 18px;
            font-weight: bold;
            margin: 15px 0;
            color: #2196F3;
        }
        label {
            display: block;
            margin: 8px 0;
        }
    </style>
</head>
<body>
    <div class="container">
        <h1>🏠 Cafeteira Inteligente</h1>
        <div class="current-time" id="time">--:--:--</div>

        <form action="/set" method="POST">
            <h3>Programar Horários</h3>
            %HORARIOS%
            <br>
            <button type="submit">Salvar Configurações</button>
        </form>

        <div id="status">
            <h3>Status dos Horários</h3>
        </div>

        <div style="margin-top: 20px;">
            <button onclick="manualNotify()" style="background:
#2196F3;">Testar Notificação</button>
            <button onclick="window.location.reload()"
style="background: #6c757d;">Atualizar Página</button>
        </div>

        <div style="margin-top: 30px; font-size: 12px; color: #666;">
            <p>Servidor Python: %PYTHON_SERVER%:%PYTHON_PORT%</p>
            <p>Status: <span
id="connectionStatus">Conectando...</span></p>
        </div>
    </div>

    <script>

```

```

function updateTime() {
    fetch('/time')
        .then(r => r.text())
        .then(t => {
            document.getElementById('time').textContent = t;
        })
        .catch(e => {
            console.error('Erro ao buscar hora:', e);
        });

    fetch('/status')
        .then(r => r.json())
        .then(data => {
            let html = '';
            data.forEach((h, i) => {
                if(h.ativo) {
                    html += `<div class="status pronto">`;
                    html += `<strong>Horário ${i+1}:</strong>
${String(h.hora).padStart(2, '0')}:${String(h.minuto).padStart(2,
'0')}`;

                    html += ` | Duração: ${h.duracao} min`;
                    html += ` | Música: ${h.musica ? '✅ ON' :
'❌ OFF'}`;

                    html += `</div>`;
                } else {
                    html += `<div class="status desligado">`;
                    html += `<strong>Horário ${i+1}:</strong>
Não programado`;

                    html += `</div>`;
                }
            });
            document.getElementById('status').innerHTML = html;

            // Verifica conexão com Python
            checkPythonConnection();
        })
        .catch(e => {
            console.error('Erro ao buscar status:', e);
        });
}

function checkPythonConnection() {
    // Tenta fazer uma requisição simples ao servidor Python

```



```

        fetch('http://%PYTHON_SERVER%:%PYTHON_PORT%', {
            mode: 'no-cors',
            method: 'HEAD'
        })
        .then(() => {
            document.getElementById('connectionStatus').textContent
= '✓ Conectado';
            document.getElementById('connectionStatus').style.color
= 'green';
        })
        .catch(() => {
            document.getElementById('connectionStatus').textContent
= '✗ Desconectado';
            document.getElementById('connectionStatus').style.color
= 'red';
        });
    }

    function manualNotify() {
        fetch('/notify?horario=1')
            .then(r => r.text())
            .then(text => {
                alert('Notificação enviada para o Python!');
                console.log('Resposta:', text);
            })
            .catch(e => {
                alert('Erro ao enviar notificação: ' + e);
            });
    }

    // Atualiza a cada segundo
    setInterval(updateTime, 1000);

    // Executa imediatamente ao carregar
    document.addEventListener('DOMContentLoaded', updateTime);
</script>
</body>
</html>
)rawliteral";

// ===== FUNÇÕES DO SERVIDOR =====
void handleRoot() {
    String html = String(INDEX_HTML);

```

```

// Gera os campos dos horários
String horariosHTML = "";
for(int i = 0; i < MAX_HORARIOS; i++) {
    horariosHTML += "<div class='horario-box'>";
    horariosHTML += "<h4>Horário " + String(i+1) + "</h4>";

    // Hora
    horariosHTML += "<label>Hora (0-23):</label>";
    horariosHTML += "<input type='number' name='h" + String(i) + "'
min='0' max='23' value='" +
        String(horarios[i].hora) + "' required>";

    // Minuto
    horariosHTML += "<label>Minuto (0-59):</label>";
    horariosHTML += "<input type='number' name='m" + String(i) + "'
min='0' max='59' value='" +
        String(horarios[i].minuto) + "' required>";

    // Duração
    horariosHTML += "<label>Duração (minutos):</label>";
    horariosHTML += "<input type='number' name='d" + String(i) + "'
min='1' max='120' value='" +
        String(horarios[i].duracaoMinutos) + "'
required>";

    // Música
    horariosHTML += "<label style='display: flex; align-items:
center;'>";
    horariosHTML += "<input type='checkbox' name='musica" +
String(i) + "' " +
        (horarios[i].tocarMusica ? "checked" : "") +
">";
    horariosHTML += "<span style='margin-left: 8px;'>Tocar música
quando pronto</span>";
    horariosHTML += "</label>";

    horariosHTML += "</div>";
}

// Substitui placeholders
html.replace("%HORARIOS%", horariosHTML);
html.replace("%PYTHON_SERVER%", String(python_server));

```

```

    html.replace("%PYTHON_PORT%", String(python_port));

    server.send(200, "text/html", html);
}

void handleSet() {
    for(int i = 0; i < MAX_HORARIOS; i++) {
        String hora_key = "h" + String(i);
        String minuto_key = "m" + String(i);
        String duracao_key = "d" + String(i);
        String musica_key = "musica" + String(i);

        if(server.hasArg(hora_key)) {
            horarios[i].hora = server.arg(hora_key).toInt();
            horarios[i].minuto = server.arg(minuto_key).toInt();
            horarios[i].duracaoMinutos =
server.arg(duracao_key).toInt();
            horarios[i].tocarMusica = server.hasArg(musica_key);
            horarios[i].ativo = true;
            horarios[i].executadoHoje = false;

            Serial.printf("Horário %d salvo: %02d:%02d por %d min,
Música: %s\n",
                        i+1, horarios[i].hora, horarios[i].minuto,
                        horarios[i].duracaoMinutos,
horarios[i].tocarMusica ? "SIM" : "NÃO");
        }
    }

    server.sendHeader("Location", "/");
    server.send(303);
}

void handleTime() {
    timeClient.update();
    server.send(200, "text/plain", timeClient.getFormattedTime());
}

void handleStatus() {
    String json = "[";
    for(int i = 0; i < MAX_HORARIOS; i++) {
        if(i > 0) json += ",";
        json += "{";
    }
}

```

```

        json += "\"ativo\":" + String(horarios[i].ativo);
        json += ",\"hora\":" + String(horarios[i].hora);
        json += ",\"minuto\":" + String(horarios[i].minuto);
        json += ",\"duracao\":" + String(horarios[i].duracaoMinutos);
        json += ",\"musica\":" + String(horarios[i].tocarMusica);
        json += "}";
    }
    json += "];";
    server.send(200, "application/json", json);
}

void handleNotify() {
    if(server.hasArg("horario")) {
        int h = server.arg("horario").toInt() - 1;
        if(h >= 0 && h < MAX_HORARIOS) {
            notificarPython(h);
            server.send(200, "text/plain", "Notificação enviada para
horário " + String(h+1));
        } else {
            server.send(400, "text/plain", "Horário inválido");
        }
    } else {
        // Teste padrão - notifica horário 1
        notificarPython(0);
        server.send(200, "text/plain", "Notificação de teste enviada");
    }
}

// ===== SETUP =====
void setup() {
    Serial.begin(115200);
    pinMode(RELE_PIN, OUTPUT);
    digitalWrite(RELE_PIN, LOW);

    // Inicializa horários com valores padrão
    for(int i = 0; i < MAX_HORARIOS; i++) {
        horarios[i].hora = 7 + i; // 7:00, 8:00, 9:00
        horarios[i].minuto = 0;
        horarios[i].duracaoMinutos = 5;
        horarios[i].ativo = false;
        horarios[i].executadoHoje = false;
        horarios[i].tocarMusica = true;
    }
}

```

```

// Conecta WiFi
Serial.println("\nConectando ao WiFi...");
WiFi.begin(ssid, password);

int tentativas = 0;
while(WiFi.status() != WL_CONNECTED && tentativas < 20) {
    delay(500);
    Serial.print(".");
    tentativas++;
}

if(WiFi.status() == WL_CONNECTED) {
    Serial.println("\n✅ WiFi conectado!");
    Serial.println("IP: " + WiFi.localIP().toString());
} else {
    Serial.println("\n❌ Falha ao conectar WiFi!");
    Serial.println("Verifique as credenciais e tente novamente.");
    while(1) delay(1000); // Loop infinito se não conectar
}

// Inicia NTP
timeClient.begin();
timeClient.update();

// Configura servidor web
server.on("/", handleRoot);
server.on("/set", HTTP_POST, handleSet);
server.on("/time", handleTime);
server.on("/status", handleStatus);
server.on("/notify", handleNotify);
server.begin();

Serial.println("✅ Servidor web iniciado!");
Serial.println("🌐 Acesse: http://" + WiFi.localIP().toString());
Serial.println("📡 Servidor Python configurado: " +
String(python_server) + ":" + String(python_port));
}

// ===== LOOP =====
void loop() {
    server.handleClient();
    timeClient.update();
}

```

```

static int ultimoDia = -1;
static unsigned long ultimaVerificacao = 0;

// Verifica novo dia a cada minuto
if(millis() - ultimaVerificacao > 60000) {
    ultimaVerificacao = millis();

    time_t rawtime = timeClient.getEpochTime();
    struct tm *ti = localtime(&rawtime);

    // Reset diário
    if(ultimoDia != ti->tm_mday) {
        ultimoDia = ti->tm_mday;
        for(int i = 0; i < MAX_HORARIOS; i++) {
            horarios[i].executadoHoje = false;
        }
        Serial.println("📅 Novo dia - flags resetados");
    }
}

int horaAtual = timeClient.getHours();
int minutoAtual = timeClient.getMinutes();

for(int i = 0; i < MAX_HORARIOS; i++) {
    if(horarios[i].ativo && !horarios[i].executadoHoje) {
        if(horaAtual == horarios[i].hora && minutoAtual ==
horarios[i].minuto) {
            Serial.println("🕒 Horário " + String(i+1) + "
atingido! Ligando cafeteira...");
            digitalWrite(RELE_PIN, HIGH);
            horarios[i].executadoHoje = true;
            releLigadoPorHorario[i] = true;
            tempoLigado[i] = millis();
        }
    }

    if(releLigadoPorHorario[i]) {
        unsigned long tempoDecorrido = (millis() - tempoLigado[i])
/ 60000;

        if(tempoDecorrido >= horarios[i].duracaoMinutos) {
            digitalWrite(RELE_PIN, LOW);

```

```

        releLigadoPorHorario[i] = false;
        Serial.println("✅ Cafeteira desligada - horário " +
String(i+1));

        // Notifica Python se música está ativada
        if(horarios[i].tocarMusica) {
            Serial.println("🎵 Enviando notificação para
Python...");

            notificarPython(i);
        }
    }
}

delay(1000);
}

```

Código Python

```

from http.server import HTTPServer, BaseHTTPRequestHandler
import json
import threading
import pygame
from urllib.parse import parse_qs
import logging
import sys
import socket
import os

# Configura logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(message)s',
    handlers=[
        logging.StreamHandler(sys.stdout)
    ]
)

class CafeHandler(BaseHTTPRequestHandler):
    def do_POST(self):
        try:
            content_length = int(self.headers['Content-Length'])
            post_data = self.rfile.read(content_length).decode('utf-8')
            params = parse_qs(post_data)

```

```

horario = int(params.get('horario', [0])[0]) + 1
hora = params.get('hora', [''])[0]

logging.info(f"☕ CAFÉ PRONTO! Horário {horario} às {hora}")

# Toca música em thread separada
threading.Thread(target=self.tocar_musica, daemon=True).start()

self.send_response(200)
self.send_header('Content-type', 'application/json')
self.end_headers()
response = json.dumps({"status": "ok", "message": "Música iniciada"})
self.wfile.write(response.encode())

except Exception as e:
    logging.error(f"Erro: {e}")
    self.send_response(400)
    self.end_headers()

def do_GET(self):
    if self.path == '/':
        self.send_response(200)
        self.send_header('Content-type', 'text/html')
        self.end_headers()

        # Obtém IP automático
        ip = self.get_local_ip()

        html = f"""
<!DOCTYPE html>
<html>
<head>
    <title>🎵 Servidor Café</title>
    <style>
        body {{ font-family: Arial; text-align: center; padding: 50px; }}
        .container {{ max-width: 500px; margin: 0 auto; }}
        .status {{ background: #4CAF50; color: white; padding: 20px;
                    border-radius: 10px; margin: 20px 0; }}
        .ip-box {{ background: #2196F3; color: white; padding: 15px;
                    border-radius: 5px; margin: 15px 0; font-family: monospace; }}
        button {{ background: #FF9800; color: white; border: none;
                    padding: 10px 20px; border-radius: 5px; cursor: pointer;
                    margin: 5px; }}
    </style>
</head>
<body>
    <div class="container">

```



```

<h1>☕ Servidor de Música da Cafeteira 🎵 </h1>
<div class="status">
  <h2>✅ ONLINE</h2>
  <p>Pronto para receber notificações</p>
</div>

<div class="ip-box">
  <h3>📶 Configuração do ESP32</h3>
  <p>No código Arduino, use:</p>
  <p><strong>IP:</strong> {ip}</p>
  <p><strong>Porta:</strong> 8080</p>
</div>

<div>
  <button onclick="testarMusica()">🎵 Testar Música</button>
  <button onclick="window.location.reload()">🔄 Atualizar</button>
</div>

<p style="margin-top: 30px;">Quando a cafeteira desligar, tocará música
aqui!</p>
</div>

<script>
  function testarMusica() {{
    fetch('/test', {{ method: 'POST' }})
      .then(r => r.json())
      .then(data => alert('Teste iniciado!'));
  }}
</script>
</body>
</html>
"""

self.wfile.write(html.encode())

elif self.path == '/test':
    threading.Thread(target=self.tocar_musica, daemon=True).start()
    self.send_response(200)
    self.send_header('Content-type', 'application/json')
    self.end_headers()
    response = json.dumps({"status": "ok", "message": "Teste iniciado"})
    self.wfile.write(response.encode())

elif self.path == '/stop':
    pygame.mixer.music.stop()
    self.send_response(200)
    self.send_header('Content-type', 'application/json')
    self.end_headers()
    response = json.dumps({"status": "ok", "message": "Música parada"})

```

```

        self.wfile.write(response.encode())

def get_local_ip(self):
    """Obtém o IP local do computador"""
    try:
        # Cria um socket temporário
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        s.connect(("8.8.8.8", 80)) # Conecta ao Google DNS
        ip = s.getsockname()[0]
        s.close()
        return ip
    except:
        return "10.181.213.28" # Fallback para o IP que você encontrou

def log_message(self, format, *args):
    pass # Suprime logs padrão

def tocar_musica(self):
    try:
        pygame.mixer.init()

        # Tenta várias músicas possíveis
        musicas_tentativas = []

        # Primeiro tenta músicas na pasta atual
        if os.path.exists("musica.mp3"):
            musicas_tentativas.append("musica.mp3")
        if os.path.exists("alarme.mp3"):
            musicas_tentativas.append("alarme.mp3")

        # Sons do Windows
        musicas_tentativas.extend([
            "C:/Windows/Media/Alarm01.wav",
            "C:/Windows/Media/Alarm02.wav",
            "C:/Windows/Media/Alarm03.wav",
            "C:/Windows/Media/Alarm04.wav",
            "C:/Windows/Media/Alarm05.wav",
            "C:/Windows/Media/notify.wav",
            "C:/Windows/Media/tada.wav",
            "C:/Windows/Media/chimes.wav"
        ])

    for musica in musicas_tentativas:
        try:
            pygame.mixer.music.load(musica)
            pygame.mixer.music.set_volume(0.7)
            pygame.mixer.music.play()
            logging.info(f"🔊 Tocando: {musica}")

```

```

import time
start = time.time()
while pygame.mixer.music.get_busy():
    if time.time() - start > 30: # Max 30 segundos
        break
    time.sleep(0.1)

pygame.mixer.music.stop()
return

except Exception as e:
    continue

# Se não encontrou arquivo, toca beep
self.tocar_beep()

except Exception as e:
    logging.error(f"Erro ao tocar música: {e}")
    self.tocar_beep()

def tocar_beep(self):
    """Toca um beep usando winsound (Windows) ou fallback"""
    try:
        import winsound
        for _ in range(3):
            winsound.Beep(1000, 300) # 1000Hz por 300ms
        import time
        time.sleep(0.2)
    except:
        # Fallback para impressão no console
        print("\n" + "🔔" * 10)
        print("🔔 CAFÉ PRONTO! 🔔")
        print("🔔" * 10 + "\n")

def descobrir_ips():
    """Descobre todos os IPs disponíveis"""
    print("\n" + "="*50)
    print("🔍 DESCOBRINDO IPs DO COMPUTADOR")
    print("="*50)

    ips_encontrados = []

    try:
        # Método 1: Usando socket
        hostname = socket.gethostname()
        ip_socket = socket.gethostbyname(hostname)
        if not ip_socket.startswith('127.'):

```

```

ips_encontrados.append(ip_socket)
print(f"📶 IP encontrado (socket): {ip_socket}")

# Método 2: Tentando conectar
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.connect(("8.8.8.8", 80))
ip_conexao = s.getsockname()[0]
s.close()

if ip_conexao not in ips_encontrados and not ip_conexao.startswith('127.'):
    ips_encontrados.append(ip_conexao)
    print(f"📶 IP encontrado (conexão): {ip_conexao}")

except Exception as e:
    print(f"⚠️ Não foi possível detectar IP automaticamente: {e}")

return ips_encontrados

def iniciar_servidor(ip='0.0.0.0', porta=8080):
    print("=" * 50)
    print("🎵 SERVIDOR DE MÚSICA DA CAFETEIRA")
    print("=" * 50)
    print(f"📶 Endereço: http://{ip}:{porta}")
    print(f"💻 Host: {socket.gethostname()}")
    print("=" * 50)
    print("\n⚠️ CONFIGURAÇÃO DO ESP32:")
    print("No código Arduino, altere esta linha:")
    print(f"\nconst char* python_server = \"{ip}\";\n")
    print("=" * 50)
    print("\n⌚ Aguardando notificações do ESP32...")
    print("Pressione Ctrl+C para parar\n")

    try:
        server = HTTPServer((ip, porta), CafeHandler)
        server.serve_forever()
    except KeyboardInterrupt:
        print("\n👋 Servidor parado pelo usuário")
    except Exception as e:
        print(f"❌ Erro: {e}")
        print("\nSoluções possíveis:")
        print("1. Use outro IP (127.0.0.1 apenas para testes locais)")
        print("2. Use outra porta (ex: 8081, 8082)")
        print("3. Execute como Administrador")
        print("4. Verifique se a porta não está em uso")

if __name__ == '__main__':
    # Verifica e instala pygame se necessário
    try:

```

```

import pygame
except ImportError:
    print("📦 Instalando pygame...")
    import subprocess
    subprocess.check_call([sys.executable, "-m", "pip", "install", "pygame"])
    import pygame

# Descobre IPs automaticamente
ips = descobrir_ips()

if ips:
    print(f"\n✅ Use este IP no ESP32: {ips[0]}")
    ip_escolhido = input(f"\n🔧 Digite o IP para usar (ou Enter para {ips[0]}): ").strip()
    if not ip_escolhido:
        ip_escolhido = ips[0]
else:
    print("\n⚠️ Não foi possível detectar IP automaticamente")
    ip_escolhido = input(f"\n🔧 Digite o IP do computador (ex: 10.181.213.28): ").strip()
    if not ip_escolhido:
        ip_escolhido = "10.181.213.28" # Seu IP

# Pede porta
porta_input = input(f"\n🔧 Digite a porta (ou Enter para 8080): ").strip()
porta = int(porta_input) if porta_input else 8080

# Cria pasta para músicas se não existir
if not os.path.exists("musicas"):
    os.makedirs("musicas")
    print("\n📁 Pasta 'musicas/' criada.")
    print("💡 Coloque arquivos MP3/WAV nela para tocar quando o café ficar pronto!")

# Inicia servidor
iniciar_servidor(ip_escolhido, porta)

```